

HourLoop

技能与时间互助平台

可行性研究与项目开发计划书

项目代号:HourLoop

文档版本:v1.0

编写日期:2026 年 4 月 30 日

项目周期:2026 年 5 月 6 日 — 6 月 20 日

编写单位:HourLoop 项目组(武汉大学)

文档说明

修订历史

版本	日期	主要变更
v0.1	2026-04-25	初始草稿, 确定平台定位
v0.5	2026-04-28	完成可行性分析与备选方案对比
v0.9	2026-04-29	补充 WBS、进度计划、风险分析
v1.0	2026-04-30	评审通过, 正式发布

文档约定

- 本文档同时承担《可行性研究报告》与《项目开发计划》两份软工标准文档的职责, 在结构上参考 GB/T 8567—88 与 GB/T 856T—88, 并依据互联网软件项目特征进行了裁剪与重组。
- 文档面向以下读者: 课程指导教师(评分用)、项目组成员(执行依据)、项目验收方(交付依据)、潜在用户代表(产品理解)。
- 文档采用半正式技术白皮书风格; 遇关键设计决策处给出依据, 遇争议处给出备选方案。
- 术语表与参考资料统一列于第 1 章末尾。

1 引言

1.1 编写目的

本文档的目的有三:其一,论证 HourLoop 项目在技术、经济、法律、社会四个维度的可行性,为项目立项提供书面依据;其二,定义项目的范围、目标、约束与验收标准,作为开发过程中需求与设计的输入;其三,给出项目的工作分解结构、人员组织安排与进度计划,用于项目执行期间的进度跟踪与资源调度。

本文档的预期读者包括:软件工程课程指导教师、项目组全体成员、项目验收答辩评委,以及任何对该项目技术方案与组织安排有兴趣的第三方读者。

1.2 项目背景

HourLoop(下称“本项目”或“系统”)是一款面向武汉大学校内的技能与时间互助 Web 应用。项目源于校内学生在课业与生活中频繁出现的双向需求:一方面,部分学生因身体不适、考研复习、求职面试等原因无法亲临课堂,但又不希望因点名缺席而影响平时分;另一方面,部分学生在特定时间段空闲且具备相应学科背景,愿意以较低成本为他人提供学习互助。当前这类需求大多在分散的微信群、QQ 群中以非正式方式撮合,存在信息散落、撮合效率低、信用难以积累、纠纷无可仲裁等问题。

本项目以“学习互助”为对外定位,通过结构化的需求与供给发布、自动化的双边匹配引擎、内部积分经济与双向评价体系,将上述非正式撮合行为收敛至单一平台,以提升撮合效率并沉淀信用。

1.2.1 项目相关方

相关方	角色描述
任务提出者	HourLoop 项目组(自命题)
开发者	HourLoop 项目组(共 6 人,武汉大学软件工程课程小组)
最终用户	武汉大学(@whu.edu.cn)在校学生
运行环境提供方	项目组自有/租赁 VPS
外部依赖方	DeepSeek(LLM 与 embedding 服务)、SMTP 邮件服务商、

相关方	角色描述
	GitHub(代码托管与 CI)

1.2.2 与其他系统的关系

本项目为独立 Web 应用,不与任何外部业务系统直接对接。仅依赖以下外部服务:DeepSeek API(用于文本 embedding 与少量语义辅助)、SMTP(用于邮箱激活与异步通知)、GitHub Actions(用于持续集成与部署)。涉及与微信生态的交互仅限“匹配成功后由用户自行通过页面展示的微信号联系”,平台不调用微信开放平台 API,亦不存储微信账号关联信息。

1.3 术语与缩写

术语 / 缩写	释义
需求方 / 供给方	在 HourLoop 中分别表示发布“求帮助”和“提供服务”两类用户角色;同一用户可在不同时间扮演不同角色
帖子(Post)	用户发布的需求或服务条目,是匹配的最小单元
匹配(Match)	系统在一对需求帖与服务帖之间建立可成单关系的过程
订单(Order)	经双方确认后由匹配生成的执行实体,承载积分流转与评价
时间币(Coin)	平台内部虚拟积分,与人民币按 1 ¥ ⇔ 10 Coin 的固定汇率挂钩(本项目周期内仅模拟充值,不通真实资金)
押金(Deposit)	接单时由系统从双方账户冻结的时间币,完成时返还,违约时罚没
Tag	用户对帖子打的语义标签,可由用户自由输入,后台进行 embedding 聚类与归一化
Embedding	由 LLM 服务对自然语言文本生成的高维向量,用于语义相似度计算
WBS	Work Breakdown Structure,工作分解结构
E2E 测试	End-to-End Testing,端到端测试,模拟真实用户操作的全链路测试
SSR / RSC	Server-Side Rendering / React Server Component,服务端渲染相关技术
DFD	Data Flow Diagram,数据流图

1.4 参考资料

- [1] GB/T 8567—1988 《计算机软件开发文件编制指南》，中国国家标准化管理委员会
- [2] GB/T 856T—1988 《项目开发计划编写规范》，中国国家标准化管理委员会
- [3] FastAPI 官方文档, <https://fastapi.tiangolo.com>
- [4] Next.js Documentation, <https://nextjs.org/docs>
- [5] PostgreSQL 16 Documentation, <https://www.postgresql.org/docs/16/>
- [6] pgvector: Open-source vector similarity search for Postgres, <https://github.com/pgvector/pgvector>
- [7] DeepSeek API 文档, <https://platform.deepseek.com>
- [8] shadcn/ui Component Library, <https://ui.shadcn.com>
- [9] OWASP Web Security Testing Guide v4.2
- [10] 项目组内部:Redis 源码分析项目结项文档(可借鉴的工程化实践)

2 项目概述

2.1 产品定位

HourLoop 定位为面向武汉大学校内的、以学习互助为主要场景的双边匹配 Web 平台。其核心命题可概括为一句话：“以时间为单位、以 tag 为索引,撮合校内学生之间的学习互助。”

产品的差异化定位由以下三点构成:其一,身份闭环:仅接受 @whu.edu.cn 邮箱注册,天然将用户范围限定在校内,降低陌生人交易的信任成本;其二,匿名优先:用户在公开广场上仅以匿名昵称、星级、完成数三项信息呈现,匹配成单后才在限定上下文中显示对方的微信/QQ 联系方式;其三,LLM 增强匹配:在传统的时间段过滤与 tag 匹配之上,引入基于 embedding 的语义相似度,允许用户用自然语言描述需求与服务,系统自动归并语义相近但表达不同的条目。

2.2 范围与边界

2.2.1 In-Scope (包含范围)

- 用户系统:基于校园邮箱的注册与激活、登录、密码找回、个人资料维护
- 内容发布:需求帖与服务帖的发布、编辑、下架,自由 tag 输入与后台聚类
- 匹配引擎:基于时间段硬过滤、tag Jaccard 相似、embedding 语义相似的综合评分
- 交易闭环:订单创建、双向确认、押金冻结与结算、违约判定
- 信用与评价:双向五星评价、文字评论、违约记录
- 内部经济:时间币账本、注册赠币、模拟充值、流水查询
- 管理后台:用户管理、订单查询、申诉处理、运营数据看板
- 演示模式:独立的演示数据集与一键身份切换,用于答辩展示

2.2.2 Out-of-Scope (不包含范围)

- 真实资金的支付与提现(本项目周期内仅以“模拟充值”按钮代替)
- 平台内部的实时即时通讯(IM)与音视频沟通(用户在成单后通过外部渠道沟通)
- 移动端原生 App(通过响应式 Web 适配移动端浏览器)
- 跨校账户体系与多租户隔离(本周期内仅服务武汉大学单一域)
- AI 自动应答、智能客服等增量能力

- 内容审核的人工初审流程(仅做基础敏感词过滤,违规内容由用户举报后人工处理)

2.3 核心功能概览

功能模块	关键能力
账户与认证	邮箱注册激活 / 登录 / 找回密码 / 匿名昵称
发布广场	需求/服务双形态发布 / 自由 tag / 时间段结构化输入 / 列表浏览
匹配与搜索	主动搜索 / 双向匹配 / 综合评分排序 / 双方确认成单
订单与履约	押金冻结 / 联系方式交换 / 双方确认完成 / 违约判定
评价与申诉	双向五星 + 文字评论 / 违约记录 / 申诉提交至管理员
积分系统	注册赠币 / 完成获得 / 模拟充值 / 流水查询 / 押金冻结
管理后台	用户管理 / 订单查询 / 申诉裁定 / 数据看板
演示模式	演示数据一键加载 / 身份切换 / 不污染生产数据

2.4 项目可交付物

2.4.1 程序

- HourLoop 前端 Web 应用源码(Next.js 项目)
- HourLoop 后端 API 服务源码(FastAPI 主服务 + 匹配引擎服务)
- HourLoop 管理后台源码(React Admin)
- 数据库迁移脚本与种子数据脚本
- Docker Compose 部署配置
- GitHub Actions CI/CD workflows 配置

2.4.2 文件

- 本文档:《HourLoop 可行性研究与项目开发计划书》
- 《HourLoop 需求规约说明书》
- 《HourLoop 概要设计说明书》
- 《HourLoop 详细设计说明书》

- 《HourLoop 测试报告》
- 《HourLoop 用户手册》
- 《HourLoop 部署与运维手册》
- 基于 OpenAPI 的接口文档(Swagger UI 自动生成)

2.4.3 服务

- 可访问的演示站点(由项目组自有 VPS 提供)
- 项目演示视频(覆盖核心用户流程)
- 答辩 PPT 与现场演示

3 需求分析

3.1 用户角色与画像

角色	画像描述
典型需求方	在校学生, 因课程冲突、身体原因、考研复习等需要他人代为出席课堂或学习互助; 对价格敏感, 对履约可靠性要求中等
典型供给方	在校学生, 有空闲时段且学科匹配, 希望以较低门槛获得校内零工收入或时间币; 对单笔收益不敏感, 对操作便捷性要求高
管理员	项目组成员或后期运营者, 负责用户管理、申诉裁定、运营数据观察

3.2 用例图与典型用例

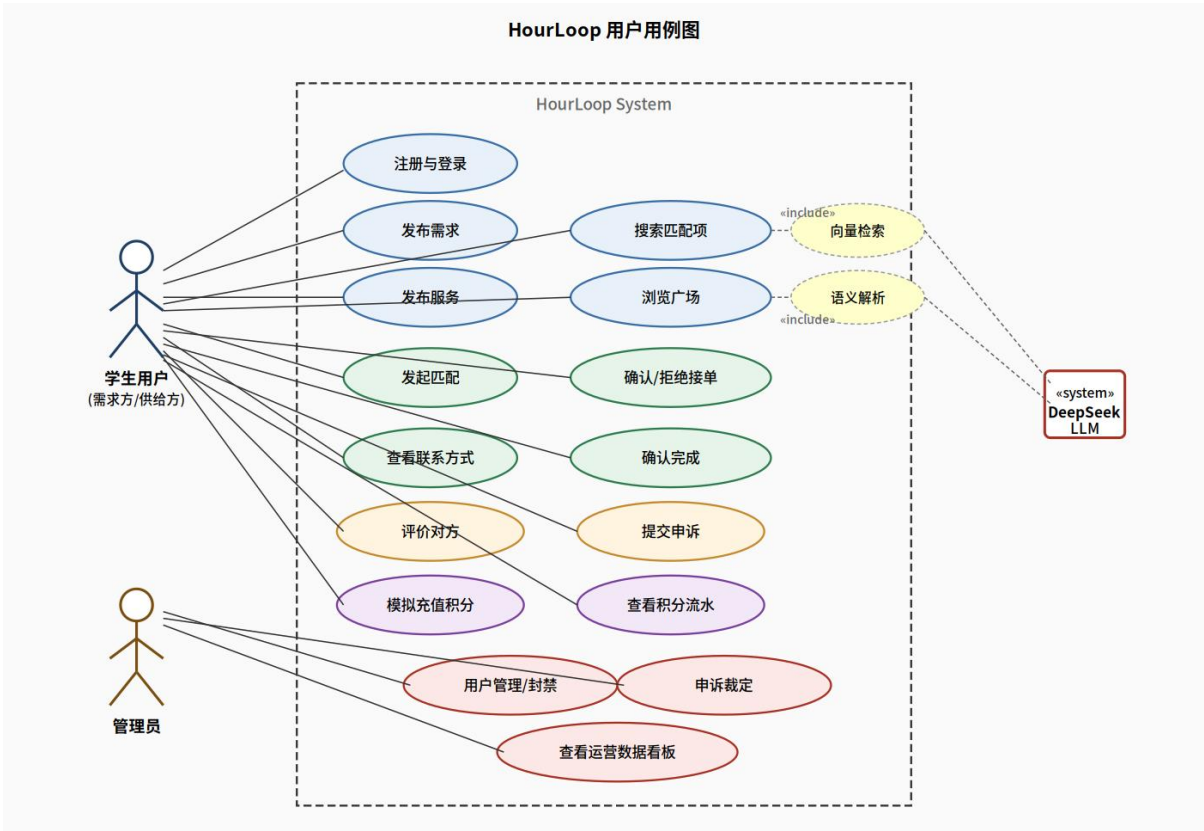


图 3-1 HourLoop 用户用例图

3.2.1 主要用例描述

以下选取三个最具代表性的用例进行说明, 完整用例描述详见《HourLoop 需求规约说明

书》。

3.2.1.1 UC-01 发布需求

项目	内容
主参与者	需求方(已登录学生用户)
前置条件	用户已通过 @whu.edu.cn 邮箱激活账户
基本路径	1. 用户进入“发布需求”页;2. 填写标题、描述、时间段、tag、价格;3. 提交;4. 系统对描述生成 embedding, 落库;5. 帖子出现在广场, 状态=open
扩展路径	3a. 时间段冲突(已发布的同时时间段未关闭需求)→ 提示用户合并或继续;5a. embedding 服务故障 → 异步重试, 降级为不参与语义匹配但仍参与 tag 匹配
后置条件	新建帖子记录, 生成 embedding 与 tag 关联, 触发广场首页缓存失效

3.2.1.2 UC-04 搜索匹配

项目	内容
主参与者	需求方或供给方
前置条件	用户已登录
基本路径	1. 用户输入查询文本与可选时间段、tag 过滤;2. 系统执行硬过滤召回候选;3. 系统执行向量召回精排;4. 系统按综合评分排序返回 Top-N;5. 用户浏览结果
扩展路径	2a. 候选为空 → 返回引导提示与“发布需求”入口;3a. embedding 服务超时 → 降级为仅 tag 评分
后置条件	前端展示匹配列表, 搜索请求被记录用于后续优化

3.2.1.3 UC-07 双向确认成单

项目	内容
主参与者	需求方与供给方

项目	内容
前置条件	需求方已发起匹配, 系统已为该匹配创建 pending 订单
基本路径	1. 系统通知供给方有新匹配请求; 2. 供给方查看请求详情, 选择接受; 3. 系统从双方账户冻结押金; 4. 双方页面解锁联系方式; 5. 订单状态 → matched
扩展路径	2a. 供给方拒绝 → 订单关闭, 需求方收到通知; 3a. 任一方时间币不足以冻结押金 → 订单作废, 通知双方
后置条件	押金已冻结, 联系方式可见, 订单进入“等待执行”状态

3.3 功能性需求 FR

功能性需求按“行动者—动作—对象”三元组组织, 每条需求均可追溯至具体用例与测试用例。

编号	需求描述	优先级
FR-01	系统应仅允许使用 @whu.edu.cn 后缀的邮箱完成注册	P0
FR-02	用户应能发布需求帖与服务帖, 并在不影响现有订单的前提下编辑或下架	P0
FR-03	用户应能输入自由文本 tag, 系统应在后台对相近 tag 进行聚类归一	P0
FR-04	用户应能以自然语言描述查询需求或服务, 系统应返回综合评分排序后的匹配列表	P0
FR-05	匹配列表应至少在前 20 项内呈现, 并附时间契合度、tag 重合度、语义相似度三项指标的可选展开	P1
FR-06	需求方发起匹配后, 系统应通知供给方, 并要求供给方在约定窗口内确认或拒绝	P0
FR-07	成单时系统应从双方账户各冻结一份押金, 完成时返还, 违约时按规则罚没	P0
FR-08	成单后双方页面应展示对方的微信/QQ; 此前账户应保持匿名	P0

编号	需求描述	优先级
FR-09	完成订单后双方应可对对方进行五星评分与文字评价, 评价不可修改	P0
FR-10	用户应能通过模拟充值按钮按 1 ¥ ↔ 10 Coin 的汇率增加自己的时间币余额	P0
FR-11	用户应能在个人中心查看完整的时间币流水, 包括来源订单引用	P1
FR-12	管理员应能查看用户列表、订单列表、申诉队列、平台运营数据	P1
FR-13	管理员应能对违规用户进行封禁, 对申诉做出“支持/驳回”的裁定	P1
FR-14	演示模式开启时, 系统应加载预设演示数据, 并允许管理员一键切换至任一演示用户身份	P2
FR-15	系统应记录关键操作日志, 供事后审计与申诉裁定使用	P1

3.4 非功能性需求 NFR

3.4.1 性能

- 匹配搜索接口在 100 并发查询、5 万条候选帖子的场景下, P95 响应时间 $\leq 800\text{ms}$
- 发布接口包含 embedding 调用, P95 响应时间 ≤ 2 秒; 若 embedding 超时, 系统应在 10 秒内异步补齐
- 广场首页(冷启动)在普通校园网带宽下 LCP ≤ 2 秒

3.4.2 可靠性

- 核心服务(前端、API、数据库)月度可用性 $\geq 99\%$, 允许每月最长 7 小时累计停机用于发布与维护
- 数据库每日 0 点自动备份, 保留最近 7 天的全量备份
- 外部 LLM 服务故障时, 系统应自动降级为仅 tag 与时间段匹配, 不影响核心用户路径

3.4.3 安全

- 用户密码必须以 `bcrypt(cost  $\geq 12$ )` 加盐哈希存储, 禁止明文或可逆加密
- 所有涉及用户操作的接口必须经过 JWT 鉴权, 关键操作(充值、提现、申诉裁定)需二次校验

- 用户的微信/QQ 联系方式以平台密钥加密存储, 仅在订单状态为 `matched` 后向特定双方解密展示
- 对外公开接口必须实施基于 IP 与用户的双层限流
- 代码不得在日志或前端控制台输出任何敏感信息(如 `token`、密钥、明文密码)

3.4.4 隐私

- 用户在广场上的呈现信息仅限:匿名昵称、平均星级、完成订单数。不显示真实姓名、学院、学号
- 成单后的联系方式展示是双方在该订单上下文中的“半透明”, 订单关闭后再次访问已不可见
- 项目组成员对生产库的访问需经过双人审计, 所有 SQL 操作记录留痕

3.4.5 可维护性

- 后端代码必须通过 Black 格式化、Ruff 静态检查;前端代码必须通过 ESLint 与 Prettier
- 核心业务逻辑(匹配评分、订单状态机、积分账本)单元测试覆盖率 $\geq 80\%$
- 所有 API 必须在 OpenAPI 中定义, Swagger UI 即可作为接口文档
- 项目结构遵循“按业务领域分包”原则, 而非“按技术分层”分包

3.4.6 兼容性

- 前端在最新两个主版本的 Chrome、Edge、Safari 与微信内置浏览器中应正常工作
- 响应式断点至少覆盖: $\leq 640\text{px}$ (手机) / $641-1024\text{px}$ (平板) / $>1024\text{px}$ (桌面)
- 后端 Python 版本 ≥ 3.11 , 数据库 PostgreSQL ≥ 16

4 系统设计

4.1 整体架构

HourLoop 采用经典的“用户层—接入层—应用层—数据层—外部服务”五层架构。前后端完全分离,前端为 Next.js SSR 应用,后端为 FastAPI 微服务集合,数据层以 PostgreSQL + pgvector 一把梭实现关系数据与向量数据的统一存储,以 Redis 承担缓存、会话与限流。整体部署于单台 VPS 之上,以 Docker Compose 编排各容器服务。

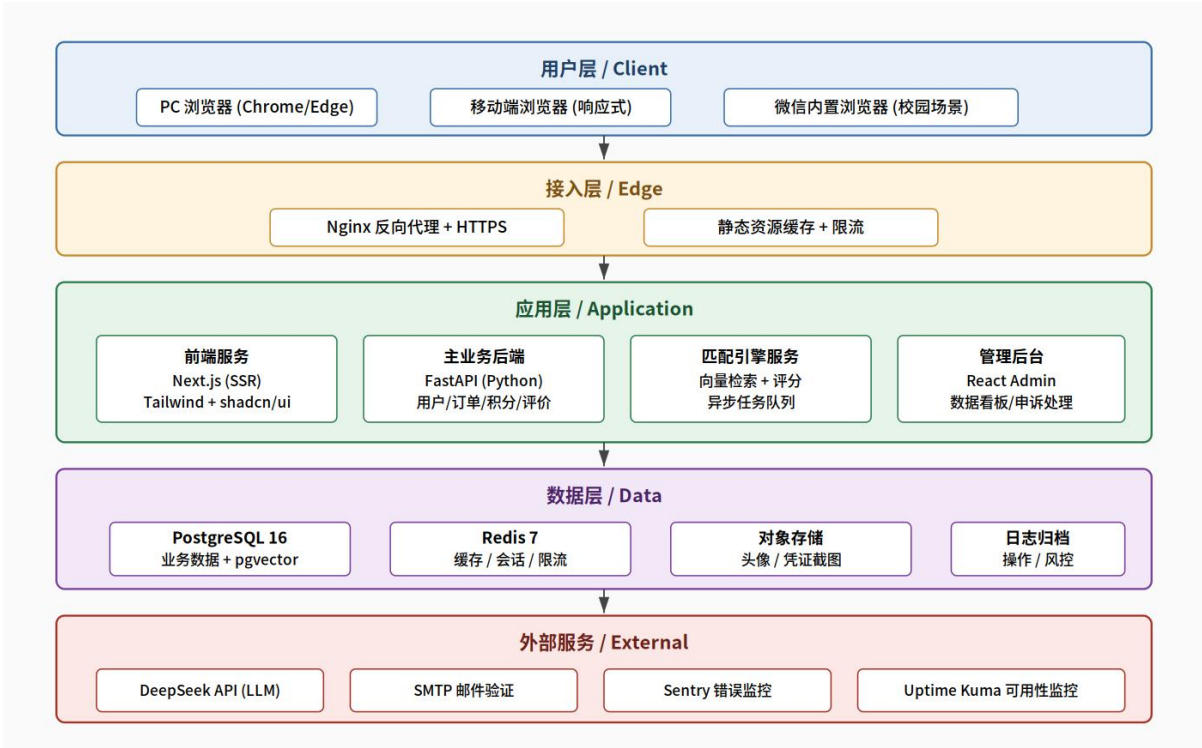


图 4-1 HourLoop 整体架构图

4.1.1 架构决策摘要

决策点	选型	主要理由
前端框架	Next.js + Tailwind + shadcn/ui	SSR 利于校园场景下的首屏速度;shadcn/ui 提供高质感默认 UI
后端语言	Python + FastAPI	FastAPI 与 LLM 生态(embedding、向量库) 集成成本最低;团队已熟
数据库	PostgreSQL 16 +	用一个数据库同时承担 OLTP 与向量检索,运

决策点	选型	主要理由
	pgvector	维成本最低
缓存	Redis 7	广场首页与匹配结果缓存、会话存储、IP 限流
LLM	DeepSeek API	国内中转免梯子, 价格低于 OpenAI 一个数量级
部署	单 VPS + Docker Compose	本期用户规模 < 1000, 无需 K8s, 降低工程复杂度
管理后台	React Admin	现成方案, 免费且功能完备, 节省自研工时

4.2 数据流

下图给出 HourLoop 顶层数据流。两类外部实体(需求方、供给方)与系统内五个处理过程(发布、向量化、匹配、订单、评价)之间的数据交换关系一目了然。三个数据存储分别是帖子与向量库、用户与积分账本、订单与日志。

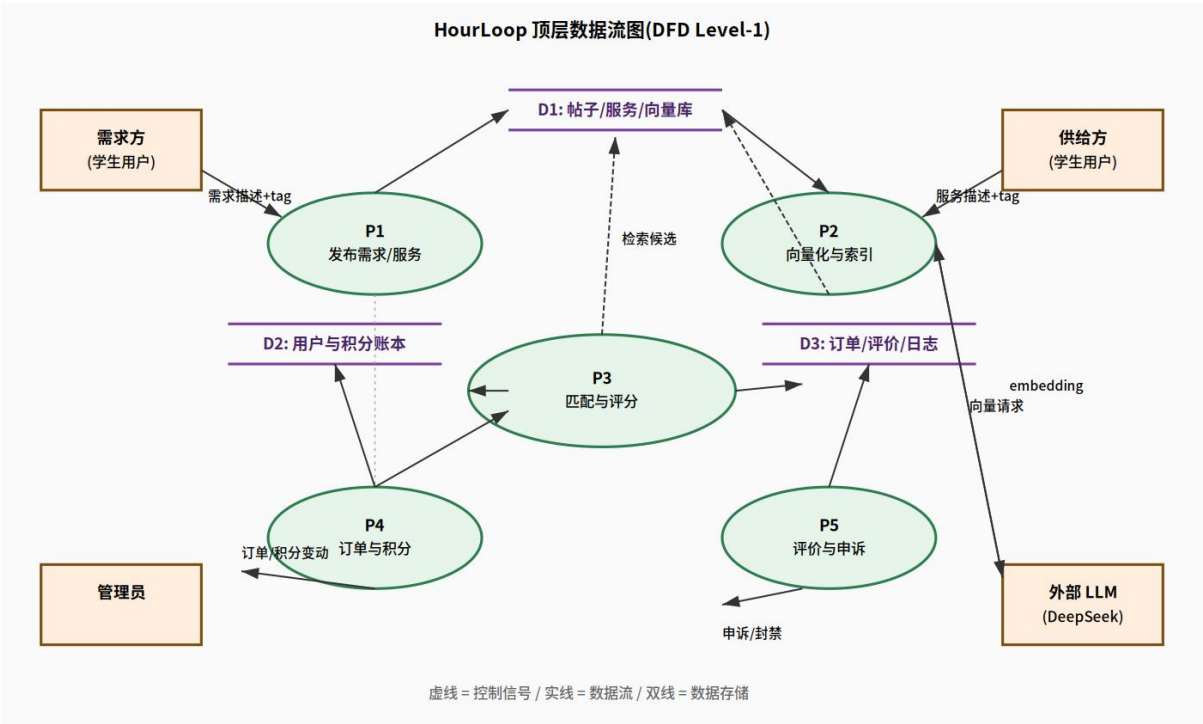


图 4-2 HourLoop 顶层数据流图

4.3 数据模型

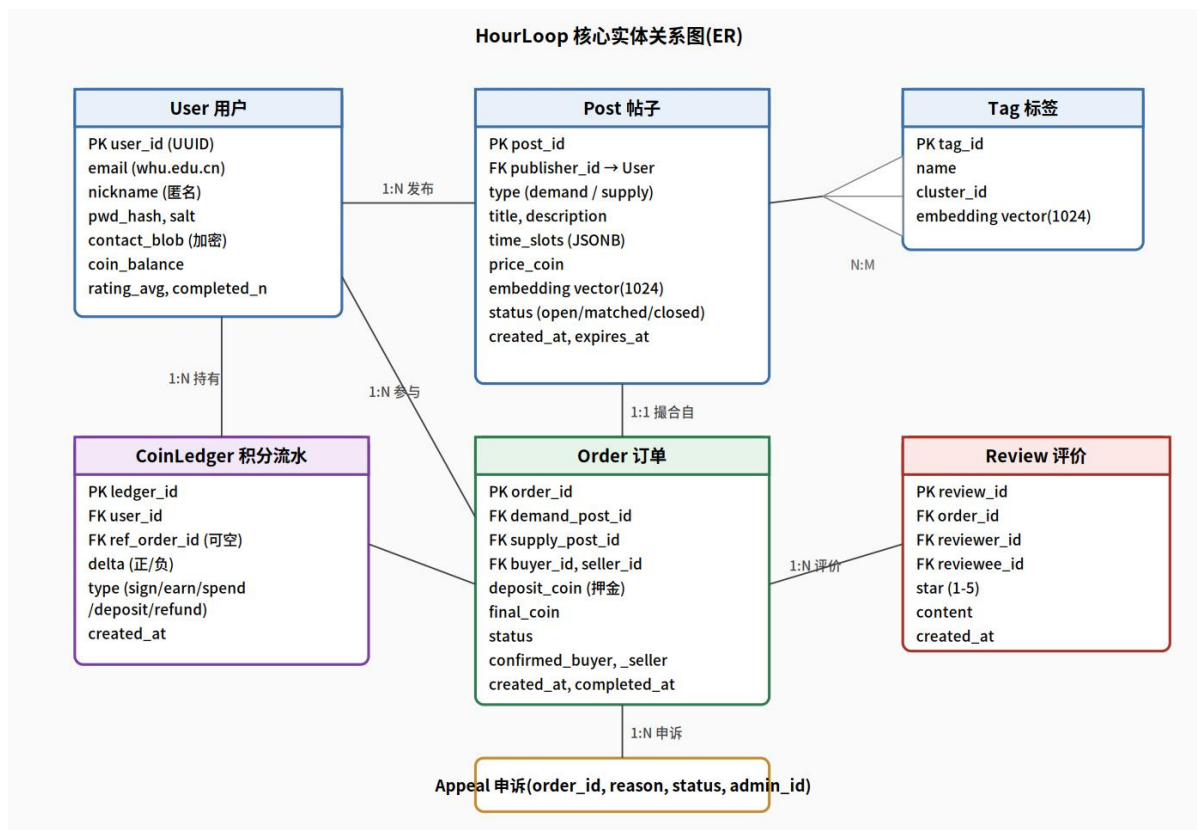


图 4-3 HourLoop 核心实体关系图

4.3.1 关键设计点

- Post 表的 embedding 字段使用 vector(1024) 类型, 由 pgvector 提供, 允许直接在 SQL 中执行 cosine_distance 检索。
- CoinLedger 表采用双记账思想: 用户余额从不直接 UPDATE, 所有变动通过插入 ledger 记录, 余额由近期累加值缓存于 User 表 (异步重算)。这样可在事故时通过流水重建账户状态。
- Order 表的 confirmed_buyer / confirmed_seller 字段独立存在, 只有当双方都为 true 才进入“已完成”状态; 任一方超时未确认即触发违约判定。
- User 表的 contact_blob 字段加密存储微信/QQ, 密钥由后端 .env 注入; 读取仅在订单中通过专用 RPC 完成, 不进入常规查询路径。
- Tag 表的 cluster_id 字段由后台定时任务 (每日凌晨) 对全量 tag 重新聚类计算, 匹配时以 cluster_id 而非 raw name 做硬过滤。

4.4 匹配算法详细设计

匹配引擎是 HourLoop 的核心。其设计要点是: 在 SQL 层完成绝大部分召回与过滤工作,

仅在评分阶段做少量 Python 计算, 从而避免将大量数据加载至应用进程内存。

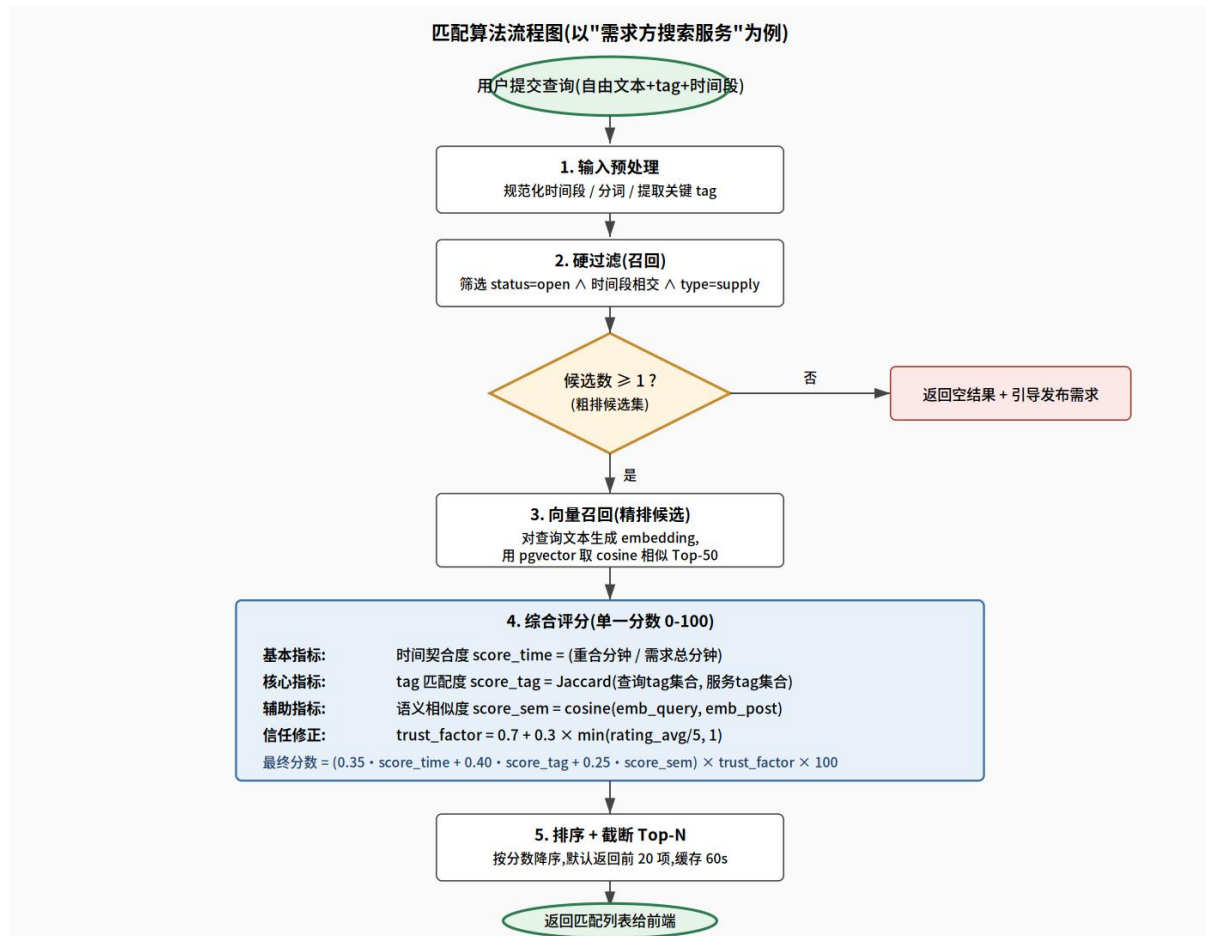


图 4-4 匹配算法流程图

4.4.1 评分公式

本期采用单一评分公式产出 0-100 的匹配分数:

$$score = (0.35 \times score_time + 0.40 \times score_tag + 0.25 \times score_sem) \times trust_factor \times 100$$

其中:

- $score_time = \text{双方时间段重合分钟数} / \text{需求方总分钟数}$, 范围 $[0, 1]$
- $score_tag = |\text{查询 tag} \cap \text{帖子 tag}| / |\text{查询 tag} \cup \text{帖子 tag}|$ (Jaccard 相似度), 范围 $[0, 1]$
- $score_sem = 1 - \text{cosine_distance}(\text{emb_query}, \text{emb_post})$ 的非负截断, 范围 $[0, 1]$
- $trust_factor = 0.7 + 0.3 \times \min(\text{供给方平均星级} / 5, 1)$, 范围 $[0.7, 1.0]$, 新用户默认 0.85

4.4.2 权重选择依据

时间是硬约束,但已在 SQL 硬过滤阶段排除完全不相交的候选,因此 `score_time` 在评分中体现的是“重合度高低”,权重 0.35;tag 是用户主动标注的最强语义信号,权重 0.40 居首;语义相似度补充了 tag 体系外的隐性匹配信号,权重 0.25;信任因子作为“乘性修正”,防止劣质供给以高单点分数刷榜。这套权重将在 W5 阶段通过项目组内部 A/B 测试做小幅调整。

4.5 部署拓扑

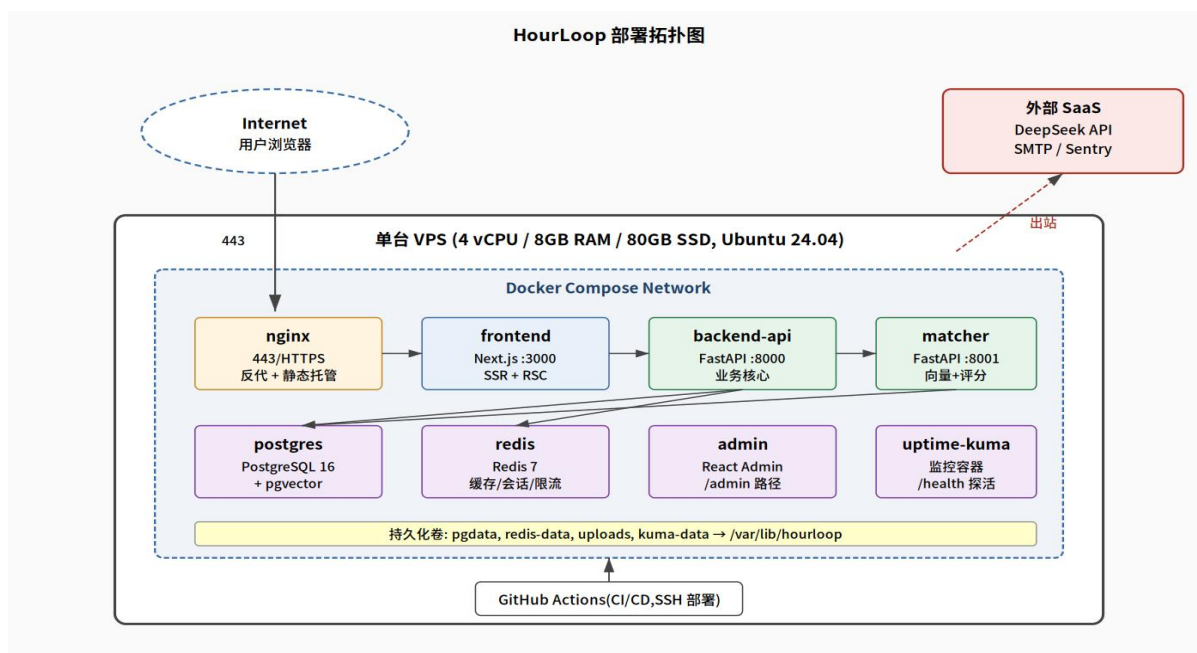


图 4-5 HourLoop 部署拓扑图

生产环境为单台 4 vCPU / 8 GB / 80 GB 的 VPS。Docker Compose 启动以下容器:nginx(443 反向代理与静态资源)、frontend(Next.js)、backend-api(FastAPI 主服务)、matcher(独立的匹配引擎服务)、postgres、redis、admin(React Admin 后台)、uptime-kuma(自托管探活)。所有持久化数据挂载于 /var/lib/hourloop。GitHub Actions 完成 CI 后通过 SSH 部署密钥触发服务器拉取与 docker compose up。

5 可行性分析

5.1 技术可行性

项目所需的全部技术栈均为成熟开源方案,团队已具备主要技能基础,关键不确定性集中在匹配引擎的工程实现而非可行性本身。

5.1.1 关键技术成熟度

技术点	成熟度评估	团队储备
Next.js + React + Tailwind	高,生态完备	需补充 1-2 名成员的 React 经验
FastAPI + SQLAlchemy + Alembic	高,Python Web 主流	团队 Python 基础充足
PostgreSQL + pgvector	中高,pgvector 已稳定 v0.7+	需 W2 阶段做技术验证
DeepSeek embedding API	中,接口稳定但不在国内外名单上	需要简单适配层封装
Docker Compose + Nginx + GitHub Actions	高,均为熟悉技术栈	项目组成员均有相关经验
Playwright E2E	高	团队已有 1 名成员有相关经验

5.1.2 关键风险点与缓解

- 匹配评分公式的权重可能不优:缓解方式是在 W5 安排 A/B 测试,若实测排序质量不达标,可在 W6 调整权重或引入 LLM rerank。
- DeepSeek API 限流或临时不可用:缓解方式是在调用层加入指数退避与本地缓存(对相同文本 24 小时内只 embed 一次),并保留“无 LLM 模式”开关。
- VPS 单点故障:缓解方式是数据库每日备份至对象存储,关键配置版本化在 Git;在课程演示阶段同时备一台备用 VPS 镜像,30 分钟内可切换。

5.2 经济可行性

本项目作为课程作业在开发阶段无商业投入, 以下经济分析按“假设产品化运营”的口径估算, 旨在论证项目本身的经济价值, 而非课程项目的实际成本核算。

5.2.1 假设产品化的支出预估(年度)

科目	年度预估(¥)	说明
VPS 与带宽	1,800	4C8G 中等配置, 国内云厂商优惠价
域名与 SSL	150	.com 域名续费, Let's Encrypt 免费证书
DeepSeek API	1,200	按 1000 DAU、每用户日均 5 次 embedding 估算
邮件 SMTP	300	选用 SendGrid/腾讯企业邮等
监控与日志	400	Sentry 免费层 + 自托管 Uptime Kuma
域名备案与合规	500	若计划商业化, 需 ICP 备案
合计	≈ 4,350	不含人力成本

5.2.2 假设产品化的收益预估(年度)

- 以武汉大学约 6 万在校生为基础, 假设 5% 注册转化率, 3000 注册用户, 其中 30% 月活 = 900 MAU
- 假设月均人均产生 4 单交易, 客单价 ¥10, 平台抽成 5% → 月收入 = $900 \times 4 \times 10 \times 5\% = ¥1,800$, 年收入 ≈ ¥21,600
- 收益/支出比 ≈ $21,600 / 4,350 \approx 4.97$, 具备造血能力
- 敏感性分析: 若 MAU 跌至 300, 年收入 ¥7,200, 仍可覆盖直接支出
- 以上数字均为假设产品化运营时的乐观估计, 本周期内不构成承诺

5.2.3 课程项目的实际成本

作为课程项目, 本项目的实际现金支出仅限于 VPS (可使用学生免费额度或项目组分摊, 合计 < ¥200) 与可选的域名 (¥60/年)。LLM 调用使用 DeepSeek 注册赠送的免费额度即可覆盖开发期。开发人力按 6 人 × 7 周 × 10 小时/周 = 420 人·小时, 不计入现金成本。

5.3 法律与合规可行性

项目涉及个人信息收集、虚拟积分流转、潜在的“代课”灰色场景,需在以下几个维度做合规设计。

5.3.1 个人信息保护

- 用户注册时仅收集邮箱与密码,不强制采集真实姓名、学号、手机号
- 微信/QQ 等联系方式属于敏感个人信息,加密存储且仅在成单上下文中向特定相对人解密展示
- 发布弹窗显式告知用户:发布内容将参与平台匹配并对其他用户可见
- 提供账户注销与个人数据导出能力(以 JSON 格式给用户下载,实现于 W5)
- 符合《个人信息保护法》《数据安全法》关于“最小必要原则”的要求

5.3.2 虚拟积分的合规

- 时间币定位为“可与现金按固定比例兑换的平台积分”,在课程项目周期内不开放真实充值与提现
- 若产品化,提现需走第三方支付聚合并依法纳税,非个人收款码;本期不涉及
- 退款机制:订单异常关闭时押金退还由系统自动完成,不留待人工

5.3.3 关于“代课”内容的合规处理

代课在国内多数高校的学生手册中明确禁止。本项目对该话题采取“平台不主动推广,但允许用户自己发布”的中庸策略:平台默认对外宣传定位为“学习互助”,在产品文案、运营推广中不出现“代课”字眼;但在用户发布的描述与 tag 中,不对此类内容做关键词屏蔽。该策略本身存在道德争议,在 7.1 节会做进一步讨论。

5.4 社会与运行可行性

5.4.1 用户接受度

- 校园场景中已存在大量基于微信群、QQ 群的非正式撮合行为,说明用户对“找人帮忙”这件事接受度高
- 仅校园邮箱准入降低了对“陌生人”的心理障碍
- 匿名机制保护用户隐私,使其敢于发布“不便公开”的需求(如代上某门冷门选修课)

5.4.2 运行环境

- 武大校园网带宽足够支撑 Web 应用流畅访问, 无特殊网络要求
- 用户自带的智能手机与 PC 即为终端, 无设备门槛
- 产品形态选择 Web App 而非小程序或 App, 跳过应用商店审核流程, 降低运行可行性的不确定性

5.5 操作可行性

- 目标用户均为大学生, 具备基本互联网操作能力, 产品交互可参考主流二手交易平台(咸鱼、转转)
- 管理员角色仅由项目组成员承担, 管理后台学习成本可由内部培训消化
- 申诉与裁定流程在用户路径中是低频操作, 文案做充分引导即可

6 备选方案比较

6.1 备选方案 1:不使用 LLM 的纯规则匹配

该方案完全去除 embedding 与语义相似度评分,匹配仅依赖时间段硬过滤与 tag 精确匹配。系统不需要调用任何 LLM 服务,对外部依赖归零,运行成本最低。

6.1.1 优势

- 外部依赖最少,系统稳定性最高
- 年度运营成本可降至 ¥3,000 以内
- 响应速度快,所有计算在 SQL 中完成, P95 < 200ms
- 用户对 tag 输入的不规整性可由“自动补全 + 限定词典”在前端解决

6.1.2 劣势

- 依赖用户标 tag 的规范性;tag 体系不完善时大量正确匹配会被漏掉
- 无法处理“语义相近但 tag 不同”的情况(如“高数辅导”与“微积分答疑”)
- 产品差异化不足,与传统校园群撮合相比缺乏故事
- 作为软件工程课程作业,缺少 LLM 集成的技术亮点

6.2 备选方案 2:微信小程序

将客户端实现为微信小程序而非 Web App,后端不变。用户通过微信扫码授权登录,绕过邮箱激活流程。

6.2.1 优势

- 用户启动路径短,无需记忆网址;微信内分享转化高
- 可直接调用微信能力(消息推送、订阅消息),实时性更强
- 用户身份与微信绑定,减少匿名带来的风险

6.2.2 劣势

- 小程序需要主体注册与备案,作为课程项目走完审核流程不现实
- 小程序开发框架与项目组成员既有技能匹配度低,学习成本高
- 微信账号身份与“@whu.edu.cn 闭环”定位冲突,无法天然限定校内

- 一旦后续要扩展至 PC 端, 仍需重写一份 Web 实现

6.3 备选方案 3: 基于公告板的纯被动撮合

不实现主动搜索与匹配引擎, 仅提供按 tag 与时间筛选的列表。用户发布需求/服务后被动等待对方浏览。系统不做评分排序, 仅按时间倒序展示。

6.3.1 优势

- 实现复杂度最低, 工时可压缩至当前估计的 50%
- 无需 embedding 与匹配引擎, 完全规避 LLM 集成的工程风险
- 产品逻辑简单, 用户教育成本低

6.3.2 劣势

- 信息匹配效率取决于用户主动浏览, 长尾需求难以被发现
- 产品形态退化为“分类信息板”, 与现有微信群相比并无显著优势
- 课程作业的技术深度不足

6.4 方案对比总结

维度	本方案	备选 1(无 LLM)	备选 2(小程序)	备选 3(公告板)
匹配质量	优	中	优	差
技术深度	高	中	高	低
运行成本	中	低	中	低
开发工时	中	中低	高	低
与团队技能匹配	高	高	低	高
课程评分潜力	高	中	高(若能完成)	中低
用户体验创新	高	中	中	低

6.4.1 选型结论

综合考虑课程作业的技术深度要求、团队技能匹配度、有限工时下的可完成性, 本项目选

择“主方案”——即 Web App + LLM embedding + 综合评分匹配引擎的组合。备选方案 1 在主方案 LLM 部分出现严重风险时可作为降级路径快速切换;备选方案 2 在产品后续扩展阶段可作为“流量入口”补充;备选方案 3 不予采纳。

7 投资与效益分析

7.1 项目投入

7.1.1 课程项目实际投入

项目	金额(¥)	说明
VPS 租赁 (7 周)	≈ 200	或使用 Azure for Students 免费额度
域名 (可选)	≈ 60	.com 一年, 可选不绑域名
LLM API	0	DeepSeek 新用户赠额覆盖
其他	0	GitHub 免费层、Sentry 免费层、Uptime Kuma 自托管
现金合计	≈ 260	可由项目组分摊
人力 (参考)	—	6 人 × 7 周 × 10 小时 = 420 人 · 小时 (无市场价)

7.1.2 若产品化的初始投入

- 注册主体与 ICP 备案: ¥0 - ¥1,200 (若使用学生组织或社团已有主体则免)
- 第三方支付通道接入 (若做真实充值/提现): ¥0 起, 按交易抽成
- 初期推广物料: ¥500 量级
- 初次部署的人力一次性投入: 已包含于课程项目工时

7.2 项目收益

7.2.1 直接经济收益 (假设产品化)

详见第 5.2.2 节估算, 在乐观假设下年度净收益约为 ¥17,000。

7.2.2 不可定量的收益

- 项目组成员获得完整的全栈项目经验, 涵盖前端、后端、LLM、DevOps、测试与文档
- 项目可作为成员未来求职的高质量作品集

- 课程作业层面预计可获得 A 等以上评分
- 若产品成功上线,可显著改善校内学习互助场景的撮合效率
- 可沉淀一套可复用的“双边市场撮合”工程实践,迁移至其他垂直场景

7.2.3 投资回收

作为课程项目,投资回收以“完成度 + 评分 + 简历价值”三项无形收益核算,不存在严格意义上的财务回收周期。若按假设产品化运营计算,以年化收支测算,月度净现金流为正,初始投入(若有)可在 6 个月内回收。

7.3 风险与敏感性

- 用户冷启动失败:若 W6 演示时未能积累足够真实用户数据,演示完全依赖种子数据,可能影响展示效果。缓解:演示模式默认开启丰富的预设数据
- LLM 服务突发不可用:DeepSeek 若临时停止服务或大幅涨价,系统应能自动降级为纯 tag 匹配。缓解:第 5.1.2 节已说明
- 法律风险升级:若校方明确发文禁止此类平台,项目应能在 24 小时内下线主域名,数据备份保留以备答辩使用
- 敏感性测算:若假设产品化的 MAU 仅达预估的 1/3,年收入跌至 ¥7,200,仍可覆盖年度直接支出 ¥4,350,经济上不亏损,但回收周期延长至 18 个月

8 项目开发计划

8.1 工作内容综述

HourLoop 项目的开发工作覆盖五个主要阶段:需求与设计(W1)、基础设施搭建(W2)、核心开发(W3-W5)、集成与测试(W5-W6)、文档与交付(W7)。各阶段之间在时间上有部分重叠,以提高资源利用率。具体的工作分解见 8.2 节,人员组织见 8.3 节,进度安排见 8.4 节。

8.2 工作分解结构(WBS)

项目的工作分解结构按“阶段—模块—任务”三层组织。一级分解为五个阶段,二级分解为各阶段下的功能模块,三级分解为模块下的具体任务。下图给出二级分解的全貌,完整的三级任务分解详见随附的项目甘特图与 Issue 跟踪。

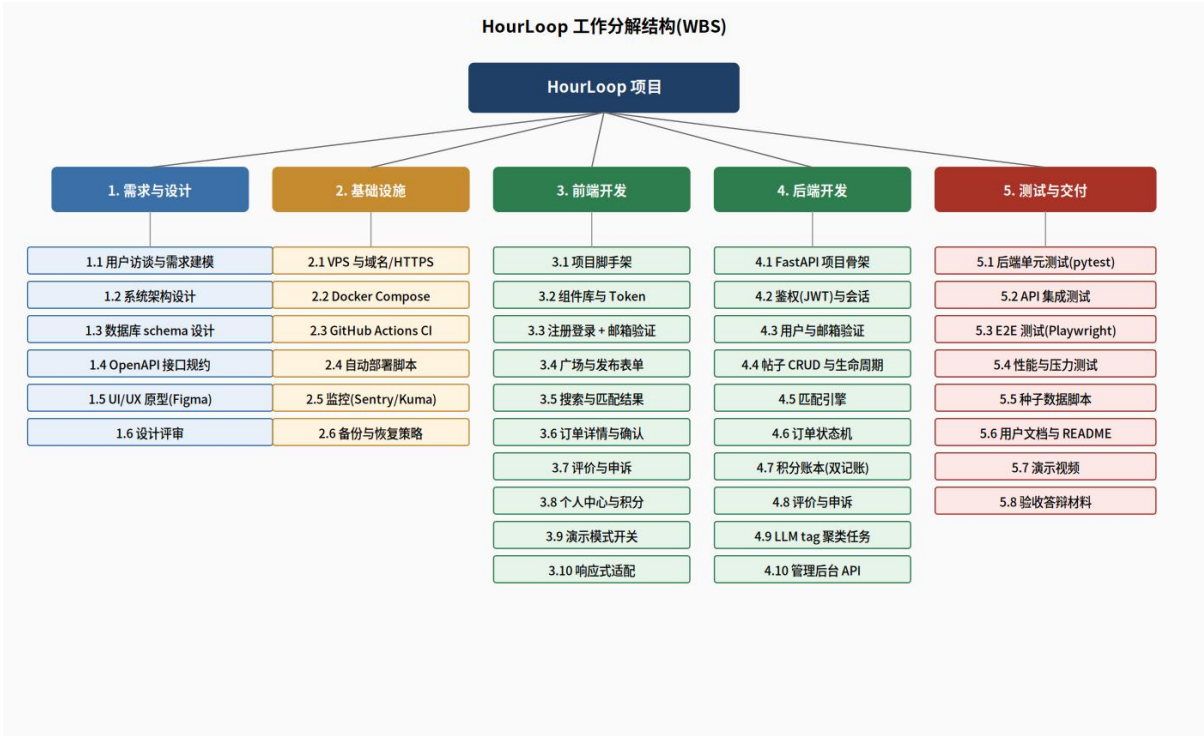


图 8-1 HourLoop 工作分解结构(WBS)

8.2.1 一级阶段说明

阶段	周次	工作要点
1. 需求与设计	W1	用户需求确认、系统设计、ER/API 定稿、UI 原型

阶段	周次	工作要点
2. 基础设施	W2	VPS、Docker Compose、CI/CD、数据库初始化
3. 前端开发	W3-W5	10 个二级模块, 从注册到演示模式开关
4. 后端开发	W3-W5	10 个二级模块, 从用户认证到管理后台 API
5. 测试与交付	W5-W7	8 个二级模块, 从单元测试到答辩材料

8.2.2 关键三级任务清单(摘录)

以下列出每个一级阶段下, 工作量较大或风险较高的关键三级任务。

8.2.2.1 阶段 1:需求与设计的关键任务

- 用户场景调研:在校内进行不少于 10 人次的非正式访谈, 验证 5 个核心使用场景
- OpenAPI 接口规约定稿:覆盖全部 P0/P1 接口, 共约 30 个, 以 YAML 形式提交至代码库
- ER 图与表结构 SQL 定稿:核心 6 张表 + 关联表, 以 Alembic 迁移脚本形式提交
- UI 原型完成度:至少覆盖广场、发布、搜索结果、订单详情、个人中心 5 个核心页面的高保真稿

8.2.2.2 阶段 2:基础设施的关键任务

- Docker Compose 编排:nginx、frontend、backend、matcher、postgres、redis、admin 七个服务的 compose.yaml 完成, 本地一键启动
- CI 工作流:推送到 main 分支自动触发后端 pytest 与前端 Lint;PR 触发 E2E smoke 测试
- CD 流水线:打 tag 触发部署, SSH 至 VPS 执行 git pull + docker compose up -d -build
- 监控接入:Sentry 项目创建并接入前端与后端;Uptime Kuma 添加 4 个核心探活点

8.2.2.3 阶段 3:前端开发的关键任务

- 项目脚手架与设计 token:Tailwind 主题配置、shadcn/ui 组件初始化、字体与色板锁定
- 注册登录:与后端的邮箱激活流程联调, 处理校园邮箱的 SMTP 延迟

- 发布表单:含富文本描述、tag 自由输入(带智能补全)、时间段结构化选择器、价格输入
- 匹配结果页:综合分数显示、三项指标可展开、空结果页引导、分页
- 演示模式开关:右下角悬浮按钮,管理员视角可见,一键切换演示用户身份

8.2.2.4 阶段 4:后端开发的关键任务

- 匹配引擎服务:作为独立 FastAPI 应用,通过内部 HTTP 与主服务通信,便于后续独立扩展
- 订单状态机:pending → matched → in_progress → completed / failed / disputed,转移条件与超时由调度任务驱动
- 积分账本:所有变动通过 INSERT INTO coin_ledger,余额 UPDATE 由触发器或应用层缓存
- LLM tag 聚类任务:每日凌晨遍历近期新增 tag,按 cosine 相似度合并至已有 cluster

8.2.2.5 阶段 5:测试与交付的关键任务

- 后端单元测试:核心评分公式、订单状态机、积分账本三个领域的测试覆盖率 $\geq 80\%$
- E2E 测试:Playwright 编写注册→发布→搜索→匹配→成单→评价的完整流程,在 CI 中无头运行
- 性能测试:用 k6 模拟 100 并发对匹配接口压测,确认 P95 $\leq 800\text{ms}$
- 种子数据:60 个演示用户、200 个帖子、80 个订单、150 条评价,带真实感的姓名与描述
- 演示视频:5 分钟,涵盖注册、发布、匹配、成单、评价、申诉、管理后台

8.3 人员组织与分工

8.3.1 团队规模与角色定义

项目组共 6 人,采用“功能模块负责人 + 横向职能”的混合模式。每位成员主导 1 个功能模块,同时在测试、文档、UI 等横向职能上承担一份兼职工作。组长负责协调与外部接口,所有成员每周参加固定例会。

8.3.2 角色职责矩阵

角色	姓名	主要职责	次要职责
项目组长 / PM	——	进度跟踪、对外接口(教师、答辩)、例会主持、风险升级	需求规约、答辩材料
前端负责人	——	前端架构、设计 token、核心页面(广场、搜索、订单)	UI/UX 原型、E2E 测试编写
后端负责人	——	后端架构、用户与帖子模块、订单状态机、积分账本	OpenAPI 文档、API 集成测试
匹配引擎负责人	——	匹配引擎服务、embedding 集成、tag 聚类任务、评分公式调优	性能测试、监控指标
DevOps 与测试负责人	——	VPS、Docker、CI/CD、监控、Playwright E2E、种子数据	部署文档、备份策略
UI 与文档负责人	——	UI 原型、视觉规范、用户文档、演示视频、管理后台前端	前端组件库维护

8.3.3 协作机制

- 代码协作:GitHub 私有仓库 + 主干开发 + Feature Branch + PR Code Review(至少 1 人审过方可合并)
- 任务跟踪:GitHub Issues + Projects 看板,WBS 三级任务转换为 Issue,以 Milestone 对应阶段
- 沟通节奏:每周一例会(同步进度、风险升级);每日异步站会(任务卡片更新);随时使用 IM 群组同步
- 代码规约:后端 Black + Ruff,前端 ESLint + Prettier,提交信息遵循 Conventional Commits
- 评审节点:W1 末设计评审、W4 末中期评审、W6 末测试评审、W7 末验收评审

8.4 进度安排

项目从 2026 年 5 月 6 日(周三)启动,至 2026 年 6 月 20 日(周六)完成,共 7 个工作周(W1-W7,合计 46 天)。考虑到组员还需兼顾其他课程,排程相对稀疏,W6-W7 留出充足缓冲。下图为周级甘特图。

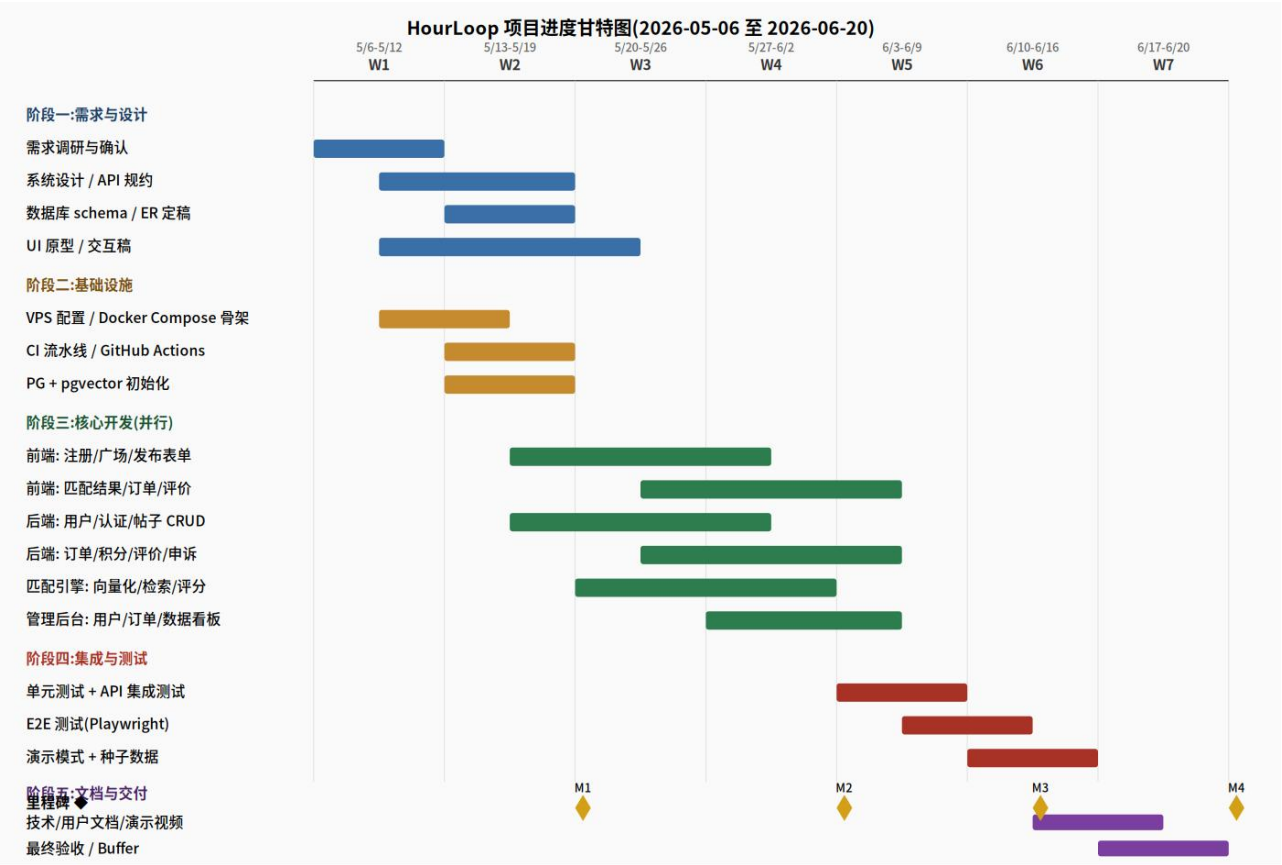


图 8-2 HourLoop 项目甘特图(2026-05-06 至 2026-06-20)

8.4.1 关键里程碑

里程碑	日期	交付物	验证方式
M1 设计冻结	2026-05-19 (W2 末)	需求规约、API 文档、ER 图、UI 原型	项目组内部设计评审通过
M2 功能完成	2026-06-02 (W4 末)	全部 P0 与 P1 功能在测试环境跑通	项目组中期评审 + Demo 演示
M3 测试完成	2026-06-16 (W6 末)	测试报告、E2E 通过、性能达标	测试评审 + 答辩彩排
M4 验收交付	2026-06-20 (W7 末)	全部文档、演示视频、部署上线	最终验收答辩

8.4.2 资源加载曲线

- W1:6 人共 60 人·小时,主要集中在需求确认与设计讨论

- W2:6 人共 60 人·小时,以基础设施搭建为主,后端与 DevOps 负责人投入最大
- W3-W5:每周 6 人共 90 人·小时(进入并行开发,工时投入达峰)
- W6:6 人共 70 人·小时,以测试与缺陷修复为主
- W7:6 人共 50 人·小时,文档收尾、演示彩排、答辩准备
- 项目总人工估算约 420 人·小时,与初始估计一致

8.5 验收标准

8.5.1 功能完整度

- FR-01 至 FR-15 全部 P0、P1 需求实现并通过对应测试
- 管理后台覆盖用户管理、订单查询、申诉处理、数据看板四个模块
- 演示模式可一键加载完整数据集并切换身份

8.5.2 测试覆盖率

- 后端核心模块(匹配评分、订单状态机、积分账本)单元测试覆盖率 $\geq 80\%$
- API 集成测试覆盖全部 P0 接口
- E2E 测试覆盖核心用户流程(注册 $\rightarrow$ 发布 $\rightarrow$ 搜索 $\rightarrow$ 匹配 $\rightarrow$ 成单 $\rightarrow$ 评价 $\rightarrow$ 申诉)

8.5.3 性能指标

- 匹配搜索接口 P95 $\leq 800\text{ms}$,在 100 并发、5 万候选规模下
- 发布接口 P95 ≤ 2 秒,首屏 LCP ≤ 2 秒
- 数据库连接池稳定不出现连接耗尽

8.5.4 部署可达性

- 一条命令在干净的 Ubuntu 24.04 服务器上完成全栈部署(以部署脚本验证)
- CI/CD 自动部署链路完整,从合并 PR 到生产环境更新 ≤ 10 分钟
- 演示站点持续可访问,Uptime Kuma 显示 99% 可用性

8.5.5 文档完备性

- 本计划书、需求规约、概要设计、详细设计、测试报告、用户手册、部署手册七份文档齐备

- API 文档由 Swagger UI 自动生成, 所有接口有描述与示例
- README 完整, 含项目简介、快速开始、贡献指南、许可证

8.5.6 演示通过

- 现场演示流畅完成核心用户流程, 无重大故障
- 能现场回答评委对架构、关键算法、风险应对的提问
- 演示视频清晰展示全部 P0 功能

8.6 风险与应对

风险描述	可能性	影响	应对措施	责任人
匹配评分公式效果不达预期	中	中	W5 安排 A/B 测试, W6 调权或引入 LLM rerank	匹配引擎负责人
DeepSeek API 突发不可用	低	高	调用层加退避与缓存, 降级为纯 tag 匹配, 保留可切换至备选 OpenRouter 通道	匹配引擎负责人
VPS 单点故障导致演示前事故	低	高	每日数据库备份, 演示前 2 小时验证备机镜像, 30 分钟内可切换	DevOps 负责人
组员因其他课业未能投入预期工时	中	中	W1 与各成员明确周工时承诺, 例会强制汇报实际投入, 人力不足时压缩 P2 功能	PM
E2E 测试在 CI 中不稳定 (flaky)	中	低	测试 retry 机制 + 关键断言分离, 稳定性差的用例转为本地手工执行	DevOps 负责人
UI 实现复杂度被低估	中	中	W1 未完成完整原型, W3 之前不允许变更	UI 负责人
注册激活邮件被校园邮箱拒收	中	中	测试早期预留 SPF/DKIM 配置时间, 准备备选 SMTP 服务商	后端负责人
代课内容引发校方负面关注	低	高	对外文案严守“学习互助”定位, 如收到通知 24 小时内下线	PM

风险描述	可能性	影响	应对措施	责任人
项目周期内成员生病/请假	中	低	关键路径任务有 Backup 责任人, 每个模块至少 2 人熟悉	PM

8.7 关键问题

以下问题在项目立项时尚无明确解, 将随开发推进逐步明确, 如出现重大变化将以变更单形式提请评审。

- 匹配评分公式中 0.35 / 0.40 / 0.25 的权重比例在实际数据下是否最优, 是否需要 LLM rerank 介入
- DeepSeek embedding 与开源 BGE 模型在校园场景下的语义匹配质量差异, 是否值得自部署模型以降低对外部 API 的依赖
- 演示阶段无真实用户的情况下, 如何让评委对“匹配效果”形成直观感受; 种子数据的设计需要充分体现匹配能力
- 申诉裁定的“鸵鸟算法”在实际纠纷下的可用性, 需在 W6 通过模拟纠纷场景检验

9 支持条件

9.1 计算机系统支持

9.1.1 开发环境

- 操作系统: macOS / Windows 11 / Ubuntu 22.04+ (开发机由组员自有, 不强制统一)
- IDE: VS Code (主推荐), JetBrains 系列 (可选)
- Node.js 版本: ≥ 20 LTS; Python 版本: ≥ 3.11
- Docker Desktop: 用于本地集成测试
- Git: ≥ 2.40 , 启用 LFS 用于设计资源

9.1.2 测试环境

- 独立的 staging VPS (2 vCPU / 4 GB / 40 GB), 配置同生产但参数减半
- staging 数据库每周从生产同步一次, 用于回归测试
- staging 不接入真实邮件 SMTP, 所有外发邮件落入 mailcatcher 容器

9.1.3 生产环境

- VPS: 4 vCPU / 8 GB RAM / 80 GB SSD, Ubuntu 24.04 LTS
- 出口带宽 ≥ 5 Mbps, 支持 IPv6
- 可选 CDN 加速静态资源 (本期内非必需)

9.2 团队需提供的工作

- 教务处或学生服务部门暂不需提供任何配合 (本期内不寻求官方背书)
- 课程指导教师按教学日程提供阶段性反馈
- 项目组成员需按周工时承诺投入, 例会出勤率 $\geq 90\%$
- 校内志愿测试用户: 招募不少于 10 名同学参与 W6 阶段的封闭测试, 提供使用反馈

9.3 由外部提供的条件

- DeepSeek 平台提供 embedding 与少量语义辅助 API, 使用其新用户赠送额度
- GitHub 提供代码托管、Actions CI 时间额度、Issues 项目管理

- Sentry 提供错误监控免费层(每月 5,000 events 额度)
- SMTP 服务商提供基本的事务性邮件发送能力
- VPS 服务商提供 IaaS 与基础运维支持(SLA 不低于 99% 月度可用性)

10 结论与建议

10.1 总体结论

综合本文档前 9 章的分析,HourLoop 项目在以下四个维度均具备可行性:技术上,所选技术栈成熟、团队储备充分、关键风险有明确缓解路径;经济上,作为课程项目的实际投入低于¥300,假设产品化后年度收支具备造血潜力;法律与社会层面,通过“学习互助”定位、匿名优先与最小化数据收集,合规风险可控;操作层面,目标用户群体的网络与设备能力完全覆盖系统需求。

项目计划在 2026 年 5 月 6 日至 6 月 20 日的 7 周窗口内,由 6 人项目组完成。WBS 已分解至三级任务,工时与里程碑安排留有 W6-W7 共两周的缓冲。验收标准在功能、测试、性能、部署、文档、演示六个维度提出明确可量化指标。

10.2 项目组结论

项目组对本项目持积极立项意见。建议按本计划书的安排立即启动 W1 阶段工作,在 M1 设计冻结(2026-05-19)之前完成需求规约与设计文档的提交,并在 M1 评审上对计划书的若干关键问题(8.7 节)给出阶段性答复。

10.3 后续工作

- 立即:细化 W1 阶段任务卡片,在 GitHub Projects 上建立 7 个 Milestone
- W1:基于本文档展开需求确认与设计,产出《需求规约说明书》《概要设计说明书》
- M1 评审:对计划书 8.7 节的关键问题给出阶段性答复,如有重大调整以变更单形式提交
- M4 验收后:整理项目复盘文档,沉淀“双边市场撮合”方向的工程实践,作为后续团队项目的参考

附录 A 接口与配置约定

A.1 RESTful API 命名约定

- 资源使用复数名词: /api/v1/posts、/api/v1/orders
- 操作通过 HTTP 方法表达: GET 查询、POST 创建、PATCH 部分更新、DELETE 删除
- 非 CRUD 动作以子资源形式表达: POST /api/v1/orders/{id}/confirm
- 分页统一使用 cursor 风格: ?cursor=xxx&limit=20
- 错误响应统一为 { code, message, details } 三字段 JSON

A.2 环境变量约定

- 所有敏感配置通过环境变量注入, 严禁硬编码
- .env.example 文件随代码版本化, 包含所有必需变量的占位符
- 生产环境的 .env 由 GitHub Secrets 注入, 不进入代码库
- 常用变量: DATABASE_URL / REDIS_URL / DEEPSEEK_API_KEY / JWT_SECRET / CONTACT_ENC_KEY

附录 B 评审与变更管理

B.1 文档评审节点

- M1 (W2 末): 本计划书 + 需求规约 + 概要设计 联合评审
- M2 (W4 末): 中期演示 + 详细设计补充评审
- M3 (W6 末): 测试报告评审
- M4 (W7 末): 最终验收答辩

B.2 变更管理

- 任何对范围、进度、人员的变更均需通过变更单提交
- 变更单包含: 变更描述、影响分析、应对方案、责任人, 经 PM 与原责任人共同签字方可生效
- 范围变更需评估对里程碑的影响, 如波及 M1-M4, 需重新评估整体计划

附录 C 文档清单

文档	完成时点	负责人
《HourLoop 可行性研究与项目开发计划书》 (本文档)	W1 起草、立项前定稿	PM
《HourLoop 需求规约说明书》	W2 末	PM + 后端负责人
《HourLoop 概要设计说明书》	W2 末	后端负责人 + 前端负责人
《HourLoop 详细设计说明书》	W4 末 (M2)	全员
《HourLoop 测试报告》	W6 末 (M3)	DevOps 负责人
《HourLoop 用户手册》	W7 上半周	UI 与文档负责人
《HourLoop 部署与运维手册》	W7 上半周	DevOps 负责人
《HourLoop 项目复盘报告》	M4 后一周	PM

—— 文档结束 ——